SECURITY AUDIT

Nami

Core - PARTIAL

FINAL REPORT

Sep 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Nami – Core – PARTIAL - Audit Report
Website	nami.fi
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/Project-Nami/Nami-Contracts/blob/14db28656de2dcffd023f450d94ad5ada90d17ee/contracts/
Resolution 1	https://github.com/Project-Nami/Nami-Contracts/tree/99ef66b8f16e9b088abc2baba7c61de118b2a19d

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	1	1				
Low	4			4		
Informational	5			5		
Governance						
Total	10	1		9		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

This report is only preliminary for the files mentioned. The full report will be delivered at a later point in time.

BatcherAccountProxyFactory

The BatcherAccountProxyFactory contract is the factory for creating BatcherAccountProxy contracts and allows each address to create one deterministic BatcherAccountProxy via the OpenZeppelin Clonable pattern.

After the version has been set-up (explanation below), users can then call the deploy function which deploys a minimal proxy pointing to the BatcherAccountProxy implementation. This implementation then remains tied to the address (msg.sender).

Once an implementation has been established, users can interact directly with it or via the execute function within this contract.

Appendix: Version Setup

BatcherAccountProxyFactory manages batcher functionality through versioned selector tables. The owner builds a mutable draft version by adding or removing facet selectors, optionally assigns an initializer, and then finalizes the draft; once finalized, users may bind their proxy to that version through setVersion. Facet dispatch is version-dependent: the proxy asks the factory for the facet mapped to msg.sig under the proxy's selected version, then delegatecalls that facet in the proxy context.

A finalized version can be paused by the owner, which blocks normal facet dispatch for proxies using that version. The owner can mark specific selectors as rescue selectors, allowing those selectors to remain callable even while the version is paused. This makes version pause a coarse emergency control, while rescue selectors preserve selected recovery paths.

When a user switches to a finalized, unpaused version, the factory records proxyVersion[proxy] = newVersionId and, if the version has an initializer, temporarily enables an init lock before calling BatcherAccountProxy.runInit. The proxy only accepts runInit from the factory and only while the factory's transient init authorization is active, so version initialization is factory-mediated and executed by delegatecall in the user proxy's storage context.

Privileged Functions

- addSelectorsToVersion
- removeSelectorsFromVersion
- setVersionInitializer
- finalizeVersion
- setVersionPaused
- setVersionRescueSelector

Invariants

INV 1: Every address can only have one proxy instance

INV 2: A version can only be set if it is finalized

INV 3: A version can only be changed in settings if it is in draft

INV 4: Users can switch between finalized, non-paused versions

Issue_01	setVersion can be called multiple times for the same version
Severity	**Informational**
Description	The setVersion function is idempotent which means it can be called multiple times for the same version. No side-effects from this behavior have been observed.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

BatcherAccountProxy

The BatcherAccountProxy contract forms the proxy entry point for users to interact with the difference facets and furthermore exposes basic functionality which does not rely on delegate-calls, such as:

- pull
- permit
- sweep
- push
- pullNFT
- pushNFT
- pushNFT
- wrapNative
- unwrapNative

The proxy is deployed as a minimal proxy via the BatcherAccountProxyFactory contract and stores the corresponding user address in the bytecode.

Privileged Functions

- multicall
- multicall
- pull
- permit
- sweep
- push
- pullNFT
- pushNFT
- wrapNative
- unwrapNative

Invariants

INV 1: Only the user or factory can trigger outgoing delegate calls

INV 2: Delegatecalls can only be triggered to a valid facet if the version is not paused

Issue_02	DoS of initialization via tokenId dusting attack (1024 limit)
Severity	**Medium**
Description	Accounts are deployed via the factory and initialized and are delegating automatically to the sink. This will however not work if an attacker pre-determined the deployed address and donates tokenIds to this address before deployment or after deployment and before initialization. It will block the initialization then.
Recommendations	Consider implementing a minimum amount for tokenId creation such that such an attack is economically unfeasible.
Comments / Resolution	Resolved by sinking all delegations.

Issue_03	Old approvals are not revoked in case of version change
Severity	**Low**
Description	Whenever a version is changed, old approvals will remain which can potentially expose unexpected risks.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_04	Users can switch between non-paused, historical versions
Severity	**Low**
Description	Whenever there are >= 1 versions, users have the flexibility to switch between versions that are not paused. This exposes ambiguous flexibility which might be used to achieve unexpected outcomes.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_05	Change of positionManager in automation and lending is not reflected in batcher approvals
Severity	**Low**
Description	Whenever the positionManager address is changed in the Lending and Automation module, this will not be reflected in the batcher and can result in unexpected side-effects.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_06	Built-in functions can clash with future facets
Severity	**Informational**
Description	The batcher has built in functions which have their own selectors and are unrelated to facets. This can expose future clashing risk if the same function selectors are added to facets.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

BatcherInitializer

The BatcherInitializer is an implementation which is used for the initialization of proxy contracts and is tied to the initializer placeholder for a version. It is set within the BatcherAccountProxyFactory.setVersionInitializer function.

It is responsible to grant approvals from the proxy to the following contracts:

- AutomationPM: Full VE approval / NAMI approval
- LendingPM: Full VE approval / NAMI approval
- wVE: Full VE approval / NAMI approval
- AutomationDiamond: NAMI approval
- LendingDiamond: NAMI approval
- VE: NAMI approval
- ogNamiStaking: ogNAMI approval

It furthermore delegates to the sink.

Appendix: Initialization

BatcherAccountProxyFactory deploys one deterministic BatcherAccountProxy per user and embeds the canonical user address into the clone's immutable arguments. The proxy's owner is the factory, while BatcherAccountProxy.user() reads the immutable user from the proxy bytecode using extcodecopy. Version selection is mediated through the factory. When a user selects a finalized batcher version, the factory records the proxy version and, if an initializer is configured, temporarily authorizes initialization before calling BatcherAccountProxy.runInit. runInit is restricted to the factory and additionally requires the factory's transient init authorization, then delegatecalls the configured initializer in the proxy context. Facet execution is therefore anchored to three initialization invariants: the proxy must be deployed by the factory, the immutable user must match the factory's users[proxy] / batchers[user] mapping, and any initializer must be executed only through the factory-controlled initialization window. Once initialized, facet helpers such as FacetBase.user() resolve the canonical immutable user through the proxy, while address(this) inside facets remains the batcher proxy that owns or custodies assets.

Privileged Functions

- none

No issues found.

ClaimerFacet

The ClaimerFacet contract is deployed as a standalone facet and invoked upon delegatecalls from proxy contracts. It exposes functionality to claim Algebra Farming Rewards and GaugeV2 rewards via the Claimer.

Privileged Functions

- none

Invariants

INV 1: Functions are only callable via delegatecall

AddressLib

The AddressLib library is a simple helper library to resolve balance, amounts and the recipient. resolveAmount lets batcher calls encode either a literal amount, the full current balance, or a percentage of the current balance in a single uint256 parameter. type(uint256).max is treated as CONTRACT_BALANCE, while values with the highest bit set are treated as percentage-encoded values using PERCENTAGE_FLAG = 1 << 255 and PERCENTAGE_MASK = PERCENTAGE_FLAG - 1. This means percentages are evaluated at execution time against the proxy's current token/native balance, not against a pre-transaction snapshot. In multicalls, every percentage resolves against the balance at that specific subcall, so prior subcalls can change later resolved amounts.

Privileged Functions

- none

Invariants

INV 1: nominal amounts must never reach 2**255

Issue_07	defaultToSender function is unused in the current scope
Severity	Informational
Description	Currently, the defaultToSender function is unused and increases the contract size for no reason.
Recommendations	Consider using or removing this function.
Comments / Resolution	Acknowledged.

BatcherConstants

The BatcherConstants contract incorporates constant variables which are uses throughout the batcher.

Privileged Functions

- none

No issues found.

BatcherProxyStorage

The BatcherProxyStorage contract incorporates a second level approval mechanism which handles approvals that are outside of the immutable batcher approval system and related to NFPM executions.

Privileged Functions

- none

Invariants

INV 1: Approvals must only be created for NFPM executions

Issue_08	secondLevelApproval function remains unused
Severity	**Informational**
Description	Currently, the secondLevelApproval function is unused and increases the contract size for no reason.
Recommendations	Consider using or removing this function.
Comments / Resolution	Acknowledged.

SwapFacet

The SwapFacet contract is deployed as a standalone facet and invoked upon delegatecalls from proxy contracts.

This facet allows for external calls to a trusted swapRouter.

Privileged Functions

- none

Invariants

INV 1: Functions are only callable via delegatecall

Issue_09	Routing over malicious route can result in unexpected balance triggers on recipient (e.g. limit order execution)
Severity	Low
Description	The SwapFacet,swap_call function processes multi-token swaps using whitelisted swap routers. However, the tokens and swapData are not validated and allow routing swaps over malicious pools. A malicious or manipulable route can cause tokenOut to land on the recipient through some path *other than* the actual swap output. For example, a resting limit order owned by the recipient gets filled during the routed call, depositing tokenOut into recipient. That inflates diffBalance, so the slippage check passes even though the swap itself delivered less than minAmountOut *struct SwapParams {* *address tokenIn;* *address tokenOut;* *uint256 amountIn;* *address swapRouter;* *bool toThis;* *uint256 minAmountOut;* *bytes swapData;* *}*
Recommendations	Consider implementing validation for tokenIn, tokenOut and swapData in the SwapFacet or the whitelisted swapRouter. Alternatively, acknowledge the issue.
Comments / Resolution	Acknowledged.

Issue_10	Residue approvals can remain of router does not fully consume approved balance
Severity	Informational
Description	When an approval is made to the swapRouter for swap execution, there is no guarantee that it fully consumes the approval made. Additionally it also creates a 1 wei approval. This leaves residue approvals to routers, which may potentially token balance drain risk if a router is unregistered in the future. *IERC20(params.tokenIn).forceApprove(params.swapRouter, params.amountIn + 1);* *// Execute swap* *(bool success,) = params.swapRouter.call(params.swapData);* *if (!success) revert FailedSwap();*
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.